2. Compile and deploy the contract with 3 as a parameter when you click the Deploy button, indicating that three proposals are available to vote on. Every time you click Deploy, the value in the box to the right of it has to be set to the number of proposals to be voted on.
3. Click the `state` button in the UI to show `0` as the value of the `state`.
4. Now click `changeState`, using `1` as a parameter, and check the value of `state`, which will show `1` for `Regs`.
5. Repeat this test for the other parameter values.

You can also see functions reverting if invalid values are given for their parameters. The Ethereum VM itself will throw an error if you give a negative value as a parameter to the `changeState()` function.

The code in listing 3.2 provides a general pattern for any smart contract that transitions through state changes in its design. In listing 3.2, the rules (for validation) for the state transitions are stated as they are in any other common code, using `if` statements. It is desirable to separate the definition of rules from the actual code of the functions to emphasize the role of the smart contract as a trust intermediator. That's what we'll do next.

## 3.2.6 Trust intermediation

Typically, verification, validation, and exceptions in problems are specified by rules to be enforced and conditions to be checked. Additionally, in a blockchain-based application, you should revert or abort any transactions that violate trust (represented by a rule) to prevent bad or unauthorized transactions from becoming part of the immutable ledger of the blockchain. This aspect is a